

YOLO Object Detection on PASCAL VOC 2007

A YOLO-v1 detector built from scratch — custom loss, target encoding, NMS, and mAP — on PASCAL VOC 2007 with a ResNet-50 backbone.

Jibril Hussaini | Python · PyTorch · torchvision · NumPy · matplotlib

Overview

Object detection requires simultaneously localising and classifying multiple objects in an image. This project implements a YOLO-v1 style detector trained on the PASCAL VOC 2007 benchmark — 9,963 images across 20 object categories. A ResNet-50 backbone pretrained on ImageNet is frozen; a small convolutional head is trained from scratch to predict a 7x7 grid of bounding boxes and class scores. The entire detection pipeline — loss function, target encoding, decoding, non-max suppression, and mean average precision — is implemented from scratch.



Detections produced by the trained YOLO model.

Goals

Build a complete, end-to-end YOLO-v1 object detection pipeline that faithfully reproduces the paper's multi-part loss and reaches a meaningful mAP on the standard VOC 2007 test set within a practical single-GPU training budget.

Objectives

- Implement the YOLO multi-part loss covering localisation, object confidence, no-object confidence, and classification terms with $\lambda_{\text{coord}} = 5$ and $\lambda_{\text{noobj}} = 0.5$.
- Encode VOC 2007 annotations into 7x7 target grids and decode predictions back to image-space boxes with non-max suppression.
- Train a ResNet-50 backbone plus convolutional head for 40 epochs using Adam with a cosine learning-rate schedule and gradient clipping.
- Evaluate detection quality with mean average precision at IoU 0.5 (mAP@0.5) on the held-out VOC 2007 test set.
- Visualise sample detections and analyse per-epoch loss and mAP trends.

Approach

The detector divides each 448x448 image into a 7x7 grid; each cell predicts two bounding boxes and 20 class probabilities, yielding a 7x7x30 output tensor. The responsible box for each ground-truth object is selected by highest IoU at each training step. The YOLO loss is implemented in

yolo_loss.py and combines four weighted terms. Training uses Adam (lr 1e-4) with a cosine schedule and gradient clipping over 40 epochs on an RTX 3080. mAP is computed periodically using a full decode-NMS-match pass over the test set.

Outcomes

- Reached mAP@0.5 of 0.324 on the VOC 2007 test set after 40 epochs, consistent with published YOLO-v1 single-GPU results at this training length.
- Final test loss of 2.655; training loss fell from 0.67 at epoch 10 to 0.08 at epoch 40, with mAP improving from 0.270 to 0.324.
- Common categories (person, car, cat, dog) are localised well; YOLO-v1 limitations appear on small or clustered objects due to the single-object-per-cell constraint.
- Full pipeline implemented from scratch: target encoding, loss, decoding, NMS, and mAP evaluation, with sample detections drawn over test images.